

Meridian

Distributed Job Scheduler

Codity.ai Placement Assignment — Submission

Tarunkumar S · ts0147@srmist.edu.in

Source code: <https://github.com/Tarunkumar060106/distributed-job-scheduler>

FastAPI · PostgreSQL · React · Docker — microservice deployment with an API gateway, Eureka-style service discovery, horizontally scalable workers, and a live dashboard.

Deliverable	Where
Source code + setup instructions	GitHub repo · README.md · Section 2 below
Architecture diagram	docs/architecture.md · Section 3
ER diagram	docs/er-diagram.md · Section 4
API documentation	docs/api.md + Swagger /docs · Section 5
Design decisions / trade-offs	docs/design-decisions.md · Section 6
Automated tests (42 passing)	backend/tests/ · Section 7

1 · What was built

A production-inspired distributed job scheduling platform. Products enqueue background jobs (emails, webhooks, reports) through a REST API; a horizontally scalable worker fleet claims and executes them with retries, timeouts, and a dead letter queue; a scheduler service handles delayed, scheduled, and cron jobs; and a web dashboard provides full operational visibility.

Core requirements — all implemented

Requirement	Implementation
Authentication & project management	JWT auth; Users → Organizations → Projects → Queues; org workspace UI
Queue configuration	Priority, per-queue concurrency limits, retry policy, pause/resume, live statistics
Job types via REST	Immediate, delayed, scheduled, recurring (cron), batch — plus idempotency keys
Worker service	Polls, atomically claims (FOR UPDATE SKIP LOCKED), executes concurrently, heartbeats, graceful SIGTERM drain
Complete lifecycle	SCHEDULED → QUEUED → CLAIMED → RUNNING → COMPLETED / FAILED → retry → DEAD_LETTER
Retry strategies	Fixed, linear, exponential — all with optional jitter; per-queue policies
Execution records	Per-attempt executions, structured logs, worker assignment, timings, metrics
Web dashboard	Queue health, worker fleet with live job attribution, job explorer, logs, retry/requeue, throughput charts

Bonus features implemented

Bonus	How
Role-based access control	OWNER / ADMIN / MEMBER / VIEWER per organization; enforced on every endpoint; one-click demo login per role
Distributed locking	Postgres advisory lock makes the scheduler an active/standby singleton (HA with zero extra infrastructure)
WebSocket live updates	Overview page receives snapshots every 2s, relayed through the API gateway
Service discovery + API gateway	Eureka-style registry (register / heartbeat lease / eviction); gateway does client-side round-robin LB with instance failover
Real job execution (Handler SDK)	Generic platform executes registered handler packs; builtin http_request (webhooks/APIs) and send_email (SMTP); fleet capability catalog

2 · Setup instructions

Quick start (Docker, one command)

```
git clone https://github.com/Tarunkumar060106/distributed-job-scheduler
cd distributed-job-scheduler
docker compose up -d --build
```

This starts 11 containers: postgres, service registry, API gateway, identity-service, job-service ×2, monitoring-service, worker ×2, scheduler, and the dashboard.

Endpoint	URL
Dashboard	http://localhost:5173
API Gateway	http://localhost:8000
Service registry catalog	http://localhost:8761/registry/services
Postgres (host)	localhost:5433

Demo accounts (seeded automatically)

One-click demo logins on the login page — all share one organization so reviewers can compare permission levels. Password for all: **demo1234**

Account	Role	Can
owner@demo.io	OWNER	Everything, including granting OWNER
admin@demo.io	ADMIN	Manage queues, projects, members
member@demo.io	MEMBER	Enqueue, retry, pause/resume
viewer@demo.io	VIEWER	Read-only (mutations return 403)

Things worth trying

```
# scale services independently
docker compose up -d --scale worker=4 --scale job-service=3

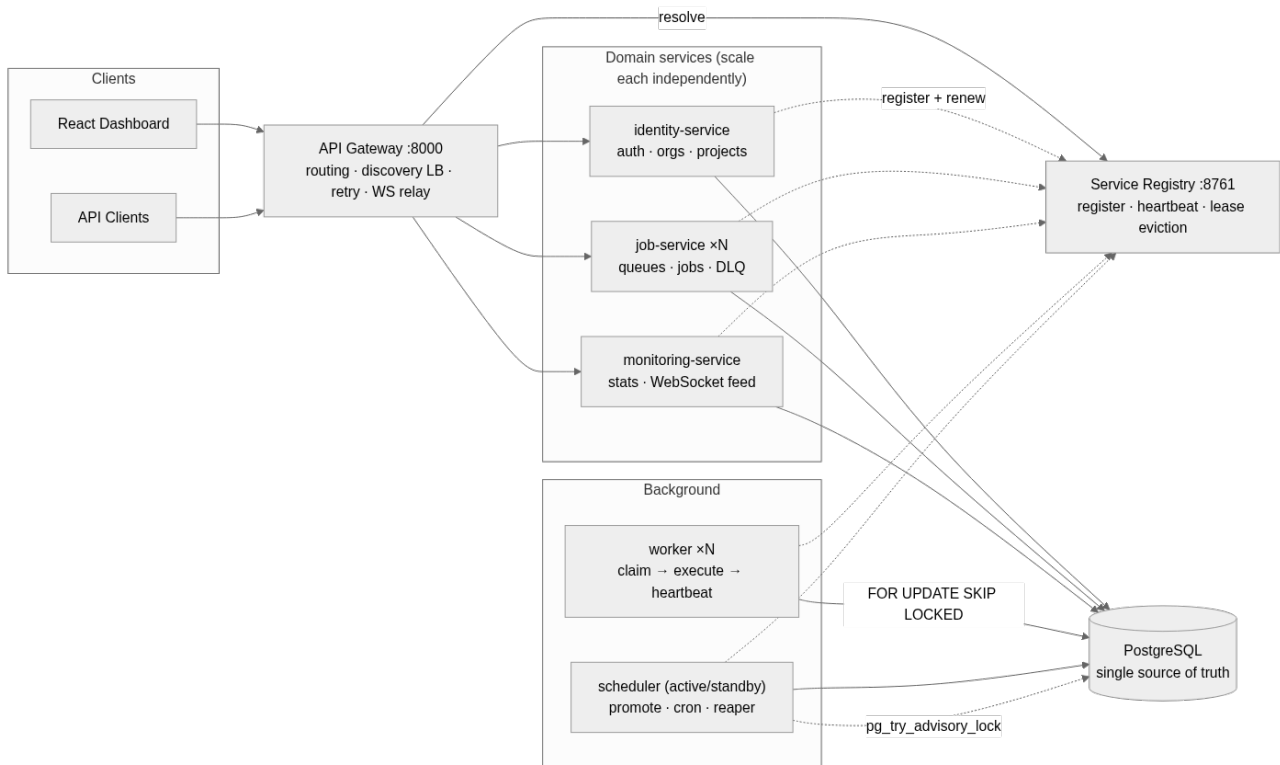
# kill a worker mid-job – the reaper reclaims and retries its jobs
docker kill distributed-job-scheduler-worker-1

# kill an API replica – the gateway fails over with zero dropped requests
docker kill distributed-job-scheduler-job-service-2
```

Run the tests

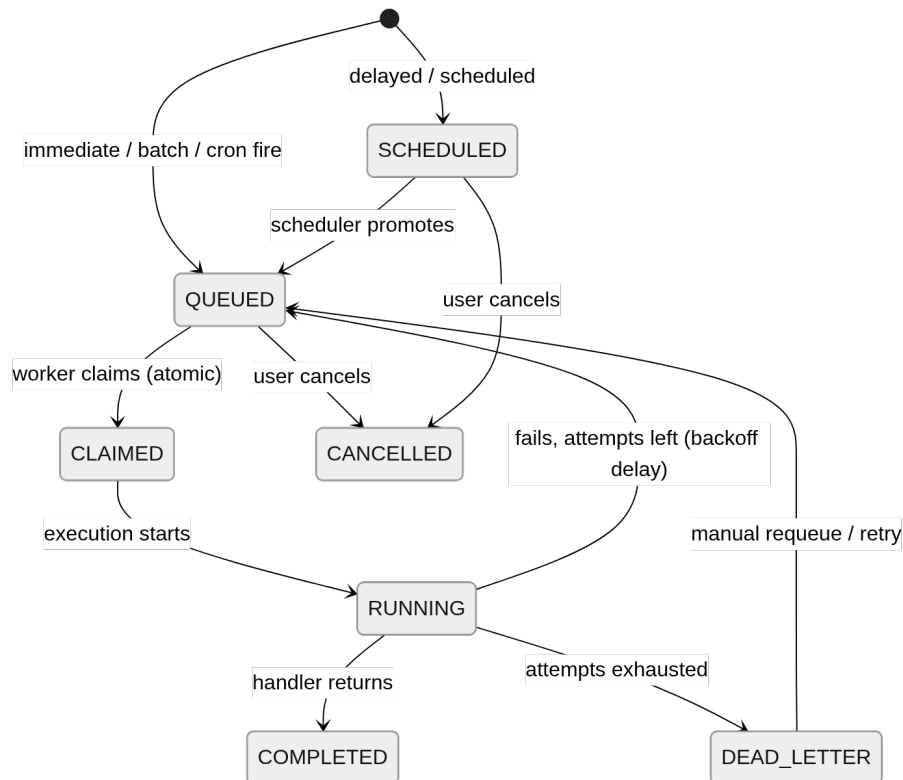
```
cd backend
python -m venv .venv && .venv/Scripts/activate
pip install -r requirements.txt
pytest tests -q # 42 passed
```

3 · Architecture



Component	Responsibility
API Gateway (:8000)	Single entry point. Longest-prefix routing to owning services; resolves healthy instances from the registry (5s cache), round-robins them, retries the next instance on connect failure; relays the dashboard WebSocket. Verified: 10/10 requests succeed while a job-service replica is killed.
Service Registry (:8761)	Eureka analogue: services register, renew a 30s lease via heartbeats, and are evicted on expiry. In-memory by design — the registry must not depend on the database it helps others find.
identity-service	Auth, users, organizations, members (RBAC), projects.
job-service (×2)	Retry policies, queues, jobs, DLQ, worker listing. Replicated behind the gateway.
monitoring-service	System stats + WebSocket live feed.
Worker fleet (×N)	Claims jobs atomically, executes handler packs in a thread pool under per-job timeouts, heartbeats every 10s, drains gracefully on SIGTERM, advertises its handler catalog to the registry.
Scheduler (active/standby)	Promotes due jobs, materializes cron templates, reaps dead workers (missed heartbeats → jobs reclaimed through the normal retry path). Advisory lock = distributed locking.
PostgreSQL	Single source of truth: queue, tenant data, execution history — transactionally consistent.

Job lifecycle

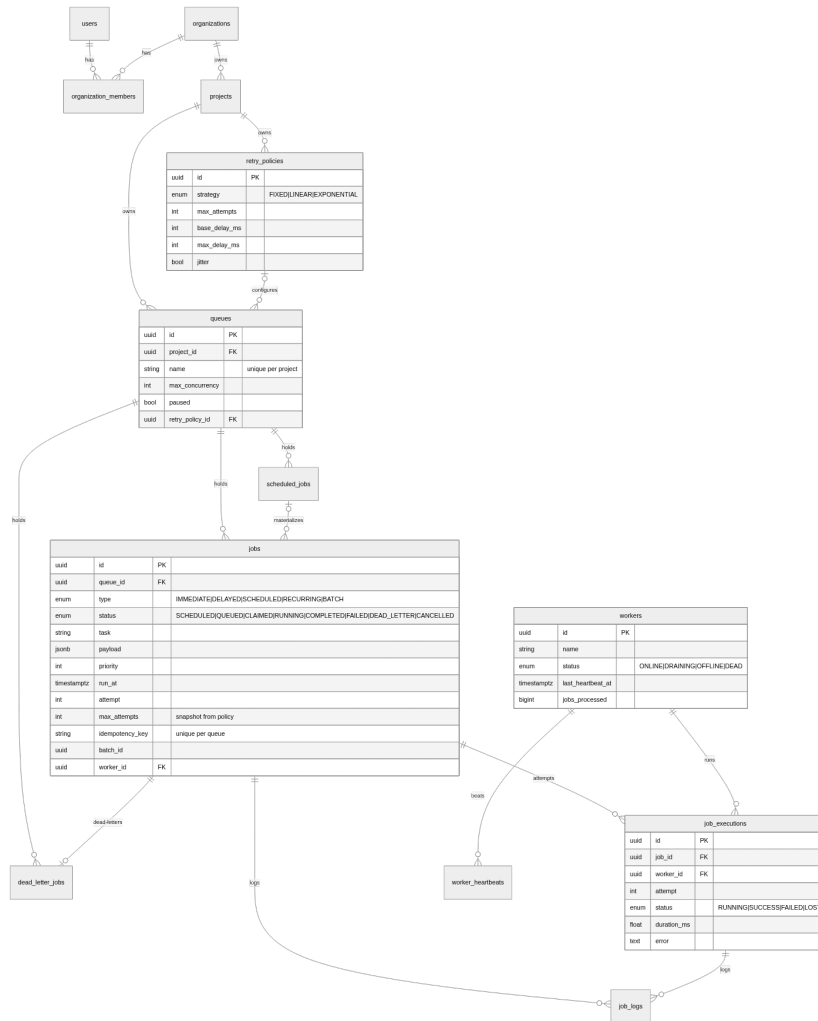


The atomic claim (core correctness guarantee)

Workers claim with a single UPDATE wrapping a SELECT ... FOR UPDATE SKIP LOCKED subquery: competing workers skip locked rows instead of blocking, so N workers drain queues in parallel and no job can ever be claimed twice. A window function caps each claim batch at the queue's remaining concurrency capacity, and a re-checked status predicate closes the race between snapshot and lock acquisition (Postgres re-evaluates predicates on the current row version under READ COMMITTED). Job distribution across workers is deliberately emergent — the competing-consumers pattern — which self-balances by capacity with no central dispatcher to fail.

Delivery semantics are at-least-once: idempotency keys deduplicate job creation, handlers declare idempotency in their contract, and every attempt is recorded for auditability.

4 · Database design



12 entities: users, organizations, organization_members, projects, queues, retry_policies, jobs, scheduled_jobs, job_executions, job_logs, workers, worker_heartbeats, dead_letter_jobs.

Keys, cascades, and indexes

Decision	Rationale
UUID primary keys	Generated by any process without coordination; no row-count leakage. Append-only tables (job_logs, worker_heartbeats) use bigserial for insert locality.
Cascade org → project → queue → job → execution → log	Natural ownership tree; deleting a tenant removes its operational data. worker_id FKs use SET NULL — deleting a worker never deletes history.
Partial index ix_jobs_claim (queue_id, run_at, priority) WHERE status='QUEUED'	The claim hot path indexes only claimable rows, staying tiny regardless of history size.
Unique (queue_id, idempotency_key)	Idempotent job creation — duplicate submits return the original job.
jobs.max_attempts snapshots the retry policy	Deliberate denormalization: editing a policy never changes jobs already in flight.
Unique dead_letter_jobs.job_id	One live DLQ entry per job; a requeued job that dies again updates its entry (regression-tested).

5 • API documentation

All traffic enters via the gateway (<http://localhost:8000>). Auth: Bearer JWT from `/api/auth/login`. Errors are structured JSON (422 validation, 401/403 auth, 404, 409 conflict). Pagination: `?page=&page_size=` → `{items, total, page, page_size}`. Interactive Swagger docs at `/docs`.

Method • Path	Purpose
POST <code>/api/auth/register</code> • <code>/login</code> • GET <code>/me</code>	Account + JWT
POST/GET <code>/api/orgs/{id}/members</code> • <code>/projects</code>	Organizations, RBAC members, projects
POST/GET <code>/api/projects/{id}/queues</code> • <code>/retry-policies</code>	Queues and retry policies
GET/PATCH <code>/api/queues/{id}</code> • POST <code>/pause</code> • <code>/resume</code> • GET <code>/stats</code>	Queue config, pause/resume, statistics
GET <code>/api/tasks</code>	Fleet handler catalog (name, JSON schema, example, idempotency)
POST <code>/api/queues/{id}/jobs</code>	Immediate / delayed (delay_seconds, run_at) / scheduled jobs; idempotency_key
POST <code>/api/queues/{id}/jobs/batch</code>	Up to 1000 jobs sharing a batch_id
POST/GET <code>/api/queues/{id}/scheduled-jobs</code> • PATCH <code>/api/scheduled-jobs/{id}/toggle</code>	Recurring (cron) templates
GET <code>/api/queues/{id}/jobs?status=&task;=&page;=</code>	Job explorer: filtered, paginated
GET <code>/api/jobs/{id}</code> • <code>/logs</code> • POST <code>/retry</code> • <code>/cancel</code>	Detail with per-attempt history, logs, actions
GET <code>/api/workers</code>	Fleet status, heartbeats, current jobs per worker
GET <code>/api/queues/{id}/dlq</code> • POST <code>/api/dlq/{id}/requeue</code>	Dead letter queue
GET <code>/api/stats/overview</code> • WS <code>/api/ws/overview</code>	System metrics + live feed
GET <code>/api/gateway/services</code>	Live service catalog from the registry

6 · Design decisions & trade-offs (summary)

The full document is docs/design-decisions.md in the repository; the major calls and their trade-offs:

Decision	Trade-off accepted
PostgreSQL as the queue (FOR UPDATE SKIP LOCKED), no Redis/RabbitMQ	Same design as Oban/pg-boss. Transactional consistency between job state and data; one datastore. Ceiling on raw throughput vs a broker; claim path isolated so a broker can be swapped in later. Polling adds ≤ 1 s latency (LISTEN/NOTIFY is the next step).
Microservices as a modular monolith deployment	One codebase, SERVICE_NAME selects routers. Independent scaling/failure per concern without premature contract duplication. Shared database with code-enforced boundaries — splitting it buys nothing at this scale.
Custom Eureka-style registry (~90 lines) instead of Consul	Demonstrates the mechanism (register/lease/evict/client-side LB) with zero infrastructure; production swap = Consul or K8s Services, only discovery.py changes.
At-least-once + idempotency, not exactly-once	Exactly-once is impossible without side-effect cooperation. Idempotency keys, idempotent-handler contract, full attempt audit trail. Crash-consumed attempts count toward max_attempts so a poisonous job can't crash-loop the fleet.
Heartbeat (10s) + reaper (30s) failure detection	A GC pause can false-positive: job re-runs (safe under at-least-once), worker rejoins on next heartbeat.
Scheduler HA via Postgres advisory lock	Active/standby with automatic failover (lock dies with the holder's connection); no ZooKeeper/etcd dependency.
Handler SDK: generic platform, pluggable business logic	Handlers are registered contracts (payload schema, idempotency, example) loaded from HANDLER_MODULES. Deterministic failures (validation, 4xx, unknown task) dead-letter immediately; transient ones retry. Timeouts are soft (Python threads can't be killed) — hard kills need process isolation, documented as next step.
Threads, not asyncio, in the worker	Handlers are arbitrary blocking code; a thread pool runs them without forcing async-awareness.

7 · Automated tests

42 tests, all passing, run against a dedicated scheduler_test database (created automatically). Focused on the correctness-critical paths rather than coverage theater:

Area	What is proven
Concurrency (test_claiming.py)	10 threads racing for 50 jobs → every job claimed exactly once; paused queues never claimed; per-queue max_concurrency caps a single claim batch; strict priority order; future jobs not claimable
Retry & DLQ (test_retry.py)	Fixed/linear/exponential backoff math incl. caps and jitter bounds; failed → requeued → dead-lettered transitions; manual requeue resets attempts; regression: requeued job dying again updates its DLQ entry
Scheduler (test_scheduler.py)	Due SCHEDULED jobs promoted; cron templates materialize exactly once and advance next_run_at; dead workers reaped and their in-flight jobs reclaimed
API contract (test_api.py)	Auth required; validation (password length, delayed-without-delay, invalid cron); idempotency dedupe; batch linkage; pagination + filtering; cancel state rules; pause/resume; RBAC: viewer 403 on mutations, outsider 403 on foreign projects; demo accounts seeded with correct roles
Handler SDK (test_handlers.py)	Pack loading; catalog metadata (JSON schema, examples); duplicate registration rejected; payload validation; permanent failures dead-letter without retries; unknown task rejected at enqueue (422); fleet task catalog endpoint
Discovery (test_discovery.py)	Registry register/heartbeat/deregister lifecycle; lease expiry eviction; gateway routing table

Beyond unit/integration tests, the full Docker stack was verified end-to-end: real HTTP jobs executed against a live external API, retries observed across two different workers, a killed worker's jobs reclaimed by the reaper, and gateway failover measured at 10/10 successful requests during an instance kill.

Thank you for reviewing — Tarunkumar S · ts0147@srmist.edu.in